Robotics 2

Lecture Notes

Flavio Massaroni

Last updated: March 11, 2026

Contents

Preface & Acknowledgments **2**

Lecture 1 – *Kinematic Calibration* **2**

1.1 Introduction 2
1.2 Calibration Procedure 3
1.3 Calibration Algorithm 4
1.4 Under- and Over-determined Systems 5
1.5 Final Comments 7

Lecture 2 – *Kinematic Redundancy* **7**

2.1 Introduction 7
2.2 Deepening the math behind kinematic redundancy 8
2.3 Inverse kinematics problem 10
2.4 Local resolution methods 13

Appendix **13**

3.1 Euler Integration Method 13
3.1.1 How it works 13
3.1.2 Application to Robotic Kinematics 14

Preface & Acknowledgments

These lecture notes are based entirely on the concepts, lectures, and slide materials provided by **Prof. Alessandro De Luca** for the *Robotics 2* course at Sapienza University of Rome (Academic Year 2025/2026).

Unless otherwise specified, all equations, algorithms, and figures are directly adapted from the official course slides. The primary reference textbook for this course is:

- B. Siciliano, L. Villani, G. Oriolo, A. De Luca, *Foundations of Robotics*, Springer, September 2025.

All original course materials are the ones disposed by the professor on the official course website:
https://www.diag.uniroma1.it/~deluca/rob2_en.php.

※ Lecture 1

1.1 Introduction

For a robot manipulator in a given configuration, the actual values of the task-space variables may slightly deviate from those computed via direct kinematics. Thus it may lead to a deviation between the end-effector pose reached in a given configuration and the pose compute via direct kinematics. Such deviations characterize the *accuracy* of a robot manipulator (its ability to reproduce precisely the commanded pose in task space). The accuracy is different from *repeatability*, which is the robot capability to reproduce exactly the same commanded pose in more attempts. It depends more on the mechanical components' and control loops' quality.

The positional accuracy error may be reduced drastically by using the **kinematic calibration procedure**. The reasons why calibration is crucial in real-world conditions can be summarized by three principal problems:

- Mechanical tolerances;
- Encoder offsets (bias which make inconsistent the "zero reference" of the robot);

- Open kinematic chain loop (which amplifies the errors distributed along the arm).

Mathematically, let $\mathbf{y}_{e,nom} = \mathbf{k}_e(\boldsymbol{\alpha}, \mathbf{a}, \mathbf{d}, \boldsymbol{\theta})$ be the nominal Direct Kinematics (DK) mapping the joint space $\mathbf{q}$ to the task space $\mathbf{y}$, with constant geometric parameters. The calibration task consists of finding the true parameter values to decrease the discrepancy between the measured pose $\mathbf{y}_e$ and the nominal pose $\mathbf{y}_{e,nom}$:

$$\Delta y_e = y_e - y_{e,nom} = \frac{\partial k_e}{\partial \alpha} \cdot \Delta\alpha + \frac{\partial k_e}{\partial a} \cdot \Delta a + \frac{\partial k_e}{\partial d} \cdot \Delta d + \frac{\partial k_e}{\partial \theta} \cdot \Delta\theta$$

In the equation, the partial derivatives represent the partial Jacobians evaluated in nominal conditions and are multiplied by the first-order variations of the DH table values.

1.2 Calibration Procedure

To rewrite the linearization in a more compact matrix form, let's define the parameter variation vector $\Delta\varphi$ (of dimension $4n \times 1$) and the Jacobian matrix Φ (of dimension $6 \times 4n$, assuming full pose data):

$$\Delta\varphi = \begin{pmatrix} \Delta\alpha \\ \Delta a \\ \Delta d \\ \Delta\theta \end{pmatrix}, \quad \Phi = \begin{pmatrix} \frac{\partial k_e}{\partial \alpha} & \frac{\partial k_e}{\partial a} & \frac{\partial k_e}{\partial d} & \frac{\partial k_e}{\partial \theta} \end{pmatrix}$$

This allows us to express the single-pose calibration equation as:

$$\Delta y_e = \Phi \Delta\varphi$$

Since a single measurement is not enough to identify all the parameters, we need to perform ℓ experiments (with $\ell \gg n$). By stacking the measured pose errors and the corresponding matrices evaluated at different nominal configurations, we obtain:

$$\overline{\Delta y_e} = \begin{pmatrix} \Delta y_{e,1} \\ \Delta y_{e,2} \\ \vdots \\ \Delta y_{e,\ell} \end{pmatrix}_{6\ell \times 1}, \quad \overline{\Phi} = \begin{pmatrix} \Phi_1 \\ \Phi_2 \\ \vdots \\ \Phi_\ell \end{pmatrix}_{6\ell \times 4n}$$

The overall calibration equation for multiple experiments becomes a linear system:

$$\overline{\Delta y_e} = \overline{\Phi}\Delta\varphi$$

In this final system:

- $\overline{\Delta y_e}$ is the vector obtained from measurements;
- $\overline{\Phi}$ is the **regressor matrix** evaluated at nominal parameters. For the system to be solvable, this matrix must have full column rank (achievable for a sufficiently large ℓ);
- $\Delta\varphi$ is the vector of unknowns we want to estimate.

1.3 Calibration Algorithm

The overall calibration equation $\overline{\Delta y_e} = \overline{\Phi}\Delta\varphi$ represents a linearized least squares problem. Since the true direct kinematics function is non-linear, this linearized formulation needs an iterative solution, dictated by the following algorithm:

$\epsilon_r = 10^{-4}, \quad i_{max} = 50$

$\varphi^{(0)} = \varphi_{nom}$

$\overline{\Phi}^{(0)} = \overline{\Phi}(\varphi^{(0)})$

$\overline{\Delta y_e}^{(0)} = \overline{\Delta y_e}(\varphi^{(0)})$

$i = 0$

if $\left\|\overline{\Delta y_e}^{(i)}\right\| \leq \epsilon_r$

  $\varphi^* = \varphi^{(i)} \quad \leftarrow$ *final solution*

else if $i \leq i_{max}$

  $\Delta\varphi^{(i)} = \left(\overline{\Phi}^{(i)}\right)^{\#} \overline{\Delta y_e}^{(i)} = \left(\overline{\Phi}^{(i)T}\overline{\Phi}^{(i)}\right)^{-1} \overline{\Phi}^{(i)T}\overline{\Delta y_e}^{(i)}$

  $\varphi^{(i+1)} = \varphi^{(i)} + \Delta\varphi^{(i)}$

  $\overline{\Phi}^{(i+1)} = \overline{\Phi}(\varphi^{(i+1)})$

  $\overline{\Delta y_e}^{(i+1)} = \overline{\Delta y_e}(\varphi^{(i+1)})$

  $i = i + 1$

else

  `disp('no convergence in` i_{max} `iterations')`

end if

Algorithm Breakdown

To better understand the iterative procedure, we can break it down into the following key steps:

1. **Initialization:** We define a tolerance threshold (ϵ_r) and a maximum number of iterations (i_{max}) to prevent infinite loops. We start by guessing that our parameters are equal to the nominal ones ($\varphi^{(0)} = \varphi_{nom}$).

2. **Convergence Check:** At each step i, we evaluate the norm of the pose error $\left\|\overline{\Delta y_e}^{(i)}\right\|$. If this error is smaller than our tolerance ϵ_r, the algorithm stops, and the current parameters are accepted as the true calibrated parameters φ^*.

3. **Least Squares Minimization:** If the error is still too large, we compute the required parameter variation $\Delta\varphi^{(i)}$. Since the matrix $\overline{\Phi}$ is rectangular (overdetermined system), we use the **Moore-Penrose pseudoinverse** $\left(\overline{\Phi}^{(i)}\right)^{\#}$. This specific mathematical operation finds the $\Delta\varphi^{(i)}$ that minimizes the squared error norm $\left\|\overline{\Phi}^{(i)}\Delta\varphi - \overline{\Delta y_e}^{(i)}\right\|^2$.

4. **Update Rule:** The calculated variation $\Delta\varphi^{(i)}$ is added to the current parameters to generate a new, more accurate guess $\varphi^{(i+1)}$. This new set contains the updated Denavit-Hartenberg parameters and the newly estimated encoder offsets (bias on θ).

5. **Re-evaluation:** The regressor matrix $\overline{\Phi}$ and the pose error $\overline{\Delta y_e}$ are re-evaluated using the newly found parameters $\varphi^{(i+1)}$, and the loop repeats.

1.4 Under- and Over-determined Systems

Different solutions may be found when considering **Under-determined** and **Over-determined** systems of equations.

In the context of the calibration algorithm, we evaluate linear systems in the standard form $Ax = b$. For our calibration, the system is $\overline{\Phi}\Delta\varphi = \overline{\Delta y_e}$. Depending on the number of equations versus the number of unknowns, we face two

different scenarios. Both are solved using the **Moore-Penrose pseudoinverse** $A^{\#}$:

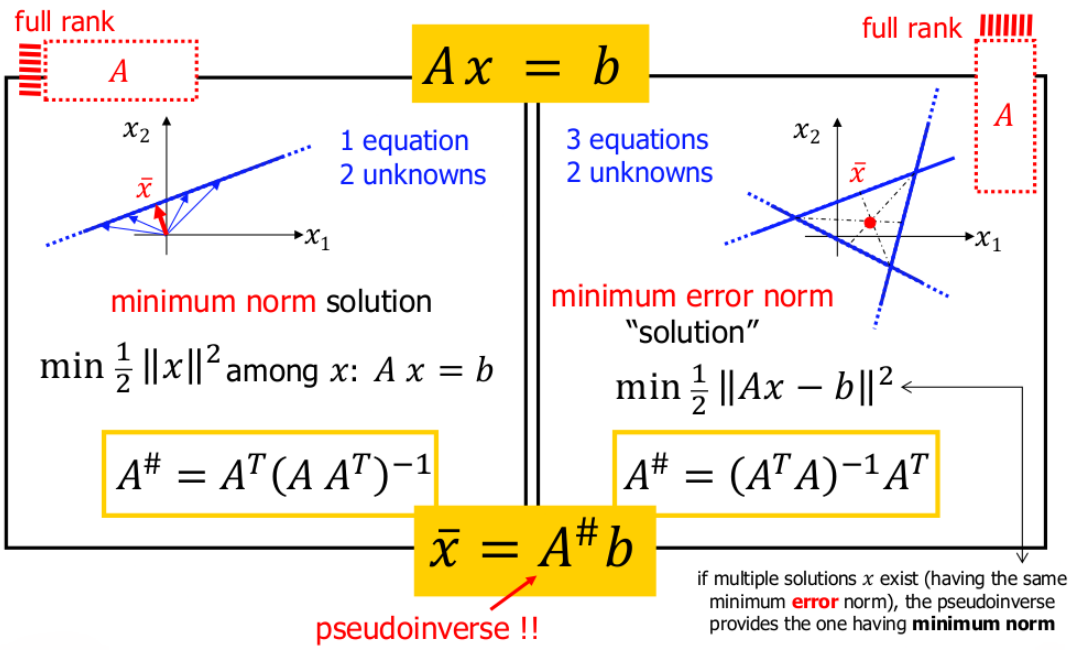


Figure 1: Geometric interpretation of under-determined and over-determined linear systems.

As illustrated in Figure 1, the pseudoinverse handles two main configurations:

- **Under-determined system (left):** There are fewer equations than unknowns (e.g., 1 equation, 2 unknowns). The matrix A has *full row rank*. Since there are infinite exact solutions, we look for the **minimum norm** solution, which minimizes $\frac{1}{2}\|x\|^2$ among all x such that $Ax = b$. The pseudoinverse used is:

$$A^{\#} = A^T(AA^T)^{-1}$$

- **Over-determined system (right):** There are more equations than unknowns (e.g., 3 equations, 2 unknowns). The matrix A has *full column rank*. Because measurements contain noise, an exact intersection usually does not exist. Instead, we look for the **minimum error norm** solution, minimizing the squared error $\frac{1}{2}\|Ax - b\|^2$. The pseudoinverse used is:

$$A^{\#} = (A^T A)^{-1} A^T$$

In both scenarios, the optimal solution is compactly written as:

$$\bar{x} = A^{\#} b$$

Note: If multiple solutions exist with the same minimum error norm, the pseudoinverse automatically provides the one having the **minimum norm**.

Connection to Calibration: Since we perform ℓ experiments where $\ell \gg n$, our calibration problem falls strictly into the **over-determined** category. This justifies why our iterative algorithm calculates the parameter variation using the left-pseudoinverse formulation: $\Delta\varphi^{(i)} = \left(\overline{\Phi}^{(i)T}\overline{\Phi}^{(i)}\right)^{-1}\overline{\Phi}^{(i)T}\overline{\Delta y_e}^{(i)}$.

1.5 Final Comments

The kinematic calibration procedure is an iterative least squared method, which starts as a non-linear problem and it is linearized by using the first-order Taylor expansion. It represent an important problem in modern Robotics, to accurate calibrate and estimate real parameters for sensors and model is foundamental to increase accuracy.

※ Lecture 2

2.1 Introduction

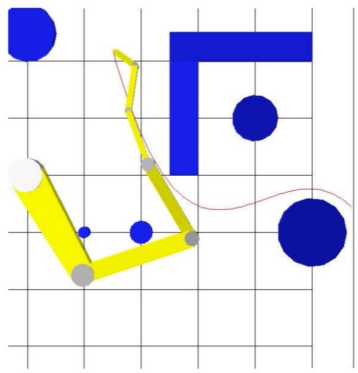
Defining the direct Kinematics of the task $y = k(q)$, which is the map between the task space and the configuration space

$$k : C \longrightarrow T$$

where $\dim(C) = n$ and $\dim(T) = m$, a robot is said to be **Kinematically redundant** for the task if $n > m$ (i.e. there are more degrees of fredom than strictly needed for executing the task). The redundancy concept is relative to a given task. A redundant robot have an *infinite number of solution* for the inverse kinematics in the regular case, and this redundancy can be used in many ways, such as:

- to avoid collision with obstacles in Cartesian space;
- to avoid kinematic singularities in Joint space;
- to stay within the admissible joint ranges;
- to increase manipulability in specified directions;

- to uniformly distribute/limit joint velocities and/or accelerations;
- to optimize energy consumption or needed motion torques;
- to minimize execution time on a given Carthesian path;
- to increase dependability with respecto to actuation faults
- ...



6R planar arm moving on a given geometric path for the E-E ($n - m = 4$)

2.2 Deepening the math behind kinematic redundancy

For an extensive comprehension of the topic, let us consider the math behind the Kinematic redundancy. The jacobian matrix J maps the joints' velocities $\dot{q} \in \mathbb{R}^n$ to the task velocities $\dot{y} \in \mathbb{R}^n$ as it follows:

$$\dot{y} = J\dot{q}$$

In a redundant situation, since $m < n$, the jacobian matrix J is a non squared matrix $m \times n$, thus the system is underdetermined and there exists infinite solutions for $\dot{q}$ for a same $\dot{y}$. The **Null Space** is the set of all the joints' velocities which produces $\dot{y} = 0$

$$N(J) = \{\dot{q} : J\dot{q} = 0\}$$

Since m < n, $dim(N(J)) = m - n > 0$, and it is said a **Non-trivial space**, which means it cannot be related to an all zeros-vector, thus it has linearly independent columns.

We can generated the **Projection Matrix P**, an $n \times n$ matrix whom image (range) concides with $N(J)$, thus $R(P) \equiv N(J)$. It means that every vector $P\dot{q}_N$, $\dot{q}_N \in \mathbb{R}^n$ is in the null space of J.

$$J \cdot P = O \implies J(P\dot{q}_N) = 0 \quad \forall \dot{q}_N$$

This mathematical statement illustrates why the projection matrix is crucial: it maps any generalized velocity vector into the null space of the Jacobian, ensuring that the resulting motion does not affect the end-effector's primary task.

The general solution for the joint velocities $\dot{q}$ that produce a desired task velocity $\dot{y}^*$ is given by:

$$\dot{q} = \dot{q}^* + P\dot{q}_N \tag{1}$$

where:

- $\dot{q}^*$ is a particular solution (e.g., the minimum-norm solution $\dot{q}^* = J^{\#}\dot{y}^*$).
- $P\dot{q}_N$ is the orthogonal projection into the null space of the Jacobian, with $\dot{q}_N$ being an arbitrary joint velocity vector.

It has a strong physical meaning. The term $P\dot{q}_N$ generates **self-motions** or **internal motions**, which consists in the robot moving and changing the overall configuration, yet the end-effector remains perfectly stationary in task space. And this is possible thanks to Kinematic Redundancy. From a control perspective, this property enables a fundamental *decoupling between the primary task and the interal configuration* of the robot. While the particual solution $\dot{q}^*$ is responsible for executing the desired task velocity $\dot{y}^*$, the null space component $P\dot{q}_N$ can be chosen freely to pursue a *secondary objective* which won't interfer with the primary task. A particular and common choice regards a manipulability measure $w(q)$, which steers the robot away from singular configurations.

$$\dot{q}_N = k\nabla w(q) \tag{2}$$

where:

$$w(q) = \sqrt{\det(JJ^T)} \tag{3}$$

and $k > 0$ is a scalar gain.

$\dot{q}_N$ can be also be chosen as a gradient-based penalty on joint proximity to physical limits, which keeps each joint away from its mechanical stops and preserves future maneuverability. Or even as a and obstacle-aware terms that push links away from detected objects in the environment, allowing the robot to reconfigure its elbow or intermediate links around an obstacle without perturbing the gripper trajectory at all.

The **general solution** is always a particular solution plus a null space component, expressed as:

$$\dot{q} = \dot{q}^* + P\dot{q}_N \tag{4}$$

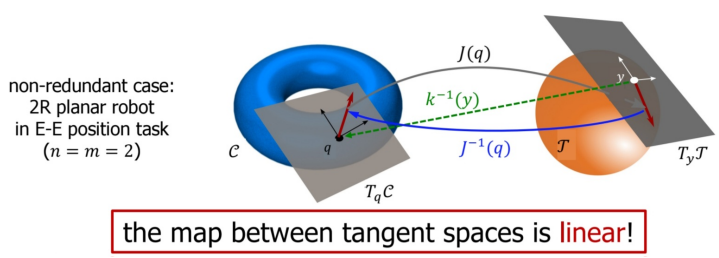
which represents the foundations of kinematic redundancy. This rests on a rigorous theoretical foundation rooted in linear algebra, since linear ($\dot{y} = J\dot{q}$ is a system of linear equations). And it is also shown that since $JP = 0$,

$$J\dot{q} = J(\dot{q}^* + P\dot{q}_N) = J\dot{q}^* + JP\dot{q}_N = \dot{y}^* + 0 = \dot{y}^* \tag{5}$$

and $P\dot{q}_N$ does not disturb the movement of the principal task. From an optimization standpoint, this structure provides a clean separation of concerns: $\dot{q}^*$ is entirely responsible for satisfying the primary task, while the remaining degrees of freedom, captured by the free vector $\dot{q}_N$, are available to optimize a secondary objective.

2.3 Inverse kinematics problem

The inverse kinematics problem for a redundant robot consists of finding $\mathbf{q}(t)$ such that $\mathbf{k}(\mathbf{q}(t)) = \mathbf{y}(t)$ for all times t. Since the robot has more degrees of freedom than strictly needed for the task, infinitely many solutions exist—even when the task vector is kept constant, $\mathbf{y}(t) = \bar{\mathbf{y}}$. The extra degrees of freedom give rise to joint displacements that are unobservable at the task level, in the sense that they do not contribute to the end-effector motion: these are precisely the self-motions introduced earlier. Solutions to the inverse kinematics problem are mainly sought at the differential (velocity) level. By considering the Jacobian matrix, which provides a linear mapping between the tangent spaces of the configuration manifold $\mathcal{C}$ and the task manifold $\mathcal{T}$, the problem is greatly simplified.

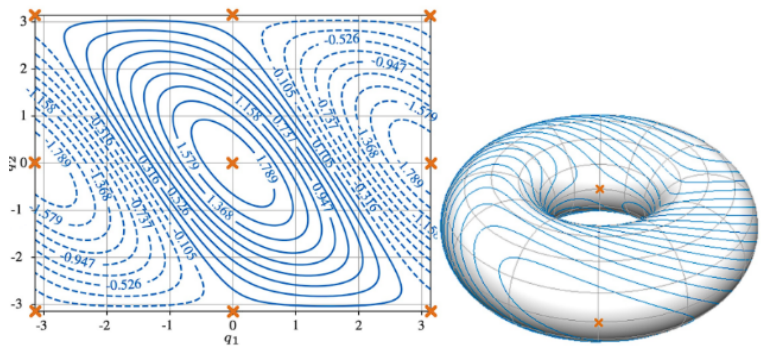


The torus $\mathcal{C} = S^1 \times S^1$ represents the configuration space of a 2R robot, where each joint varies in the interval $[-\pi, \pi]$ with periodic boundary conditions. The plane $T_{\mathbf{q}}\mathcal{C}$ represents the tangent space at a given configuration $\mathbf{q}$, which contains the joint velocity vectors $\dot{\mathbf{q}}$.

On the right side, $\mathcal{T} = \mathbb{R}^2$ represents the task space (e.g., the position of the end-effector in the Cartesian plane), and $T_{\mathbf{y}}\mathcal{T}$ is its corresponding tangent space containing the task velocities $\dot{\mathbf{r}}$ (or $\dot{\mathbf{y}}$).

The Jacobian matrix $\mathbf{J}(\mathbf{q})$ acts as a linear mapping between these tangent spaces, $T_{\mathbf{q}}\mathcal{C} \longrightarrow T_{\mathbf{y}}\mathcal{T}$. Since the robot is non-redundant ($n = m = 2$), the inverse mapping is uniquely defined by $\mathbf{J}^{-1}(\mathbf{q})$, provided the robot is not in a singular configuration.

Instead, in the following image it is described a redundant case of the same 2R planar robot, with the task described only for the ee position component p_x, thus $n = 2$ and $m = 1$.



In this case we can solve the IK problem through the *general redundancy resolution*, which is an optimization problem that can be solved offline (globally) or online (locally). The objective funcion is a general function of the state space ($\mathbf{q}\dot{\mathbf{q}}$), and the goal is to find a trajectory $\mathbf{q}(t)$ and $\dot{\mathbf{q}}$.

Note: We are in the time domain, $\mathbf{q}(t)$ and $\dot{\mathbf{q}}$ are joints position and velocity with respect to t. $H(\mathbf{q}(t)$ and $\dot{\mathbf{q}})$ is a **Goal Function**, (that we can consider as a Cost function): it evaluates how good is a configuration or a movement. The function H can be seen in a more global way:

- **Minimizing the energy consumption**: $H(\mathbf{q}, \dot{\mathbf{q}}) = \frac{1}{2}\dot{\mathbf{q}}^T\mathbf{M}(\mathbf{q})\dot{\mathbf{q}}$
- **To get away from singularities**: $H(\mathbf{q}) = \sqrt{\det(\mathbf{J}\mathbf{J}^T)}$
- **To avoid an Obstacle**: $H(\mathbf{q})$ can be the inverse of the distance between the robot arm and the obstacle;

To better understand how this optimization is carried out, we introduce the two main methodologies:

- **Global methods (Offline)**: This approach looks at the problem in its entirety. Given a desired task trajectory $\mathbf{y}(t)$ and an initial configuration $\mathbf{q}(t_0)$, finding an evolution over the entire time window $[t_0, t_0 + T]$ involves a functional of the type:

$$\int_{t_0}^{t_0+T} H(\mathbf{q}, \dot{\mathbf{q}})dt \tag{6}$$

  which has to be minimized over the whole trajectory $\mathbf{q}(t)$.

  Why is it difficult? The mathematics behind this relies on the Calculus of Variations and produces a Two-Point Boundary Value (TPBV) problem, resulting in highly non-linear differential equations. Consequently, it cannot be computed in real-time while the robot is moving; the entire optimal trajectory must be pre-calculated offline before execution.

- **Local methods (Online)**: This is the approach used in the vast majority of real-world robotic applications. Instead of optimizing over the entire journey, the controller evaluates only the current time step $t = kT_s$ (where T_s is the sampling time, e.g., 1 ms). The objective becomes much simpler: given the instantaneous task velocity $\dot{\mathbf{y}}$ required right now, what is the joint velocity $\dot{\mathbf{q}}$ that realizes the task while simultaneously optimizing H at this exact instant?

  Why is it easier? Mathematically, this reduces to a Linear-Quadratic (LQ) optimization problem, which modern processors can solve instantaneously. Once the optimal joint velocity $\dot{\mathbf{q}}(kT_s)$ is found, the joint positions are updated using Euler integration, see Appendix 3.1:

$$\mathbf{q}((k+1)T_s) = \mathbf{q}(kT_s) + T_s\dot{\mathbf{q}}(kT_s) \tag{7}$$

  where k is an integer index ($k = 0, 1, 2, \dots$) representing the current control cycle.

  To better understand its physical meaning, we can break down the equation as follows:

  - $\mathbf{q}((k+1)T_s)$: The joint configuration that the robot must reach at the **next time step**.
  - $\mathbf{q}(kT_s)$: The joint configuration at the **current time instant**.
  - $T_s\dot{\mathbf{q}}(kT_s)$: The small (angular) **displacement** the joints will undergo. This term is computed by taking the optimal velocity calculated right now ($\dot{\mathbf{q}}$) and multiplying it by the duration of the time interval until the next update (T_s).

2.4 Local resolution methods

Three are the classes of methods for solving $J(q)\dot{q} = \dot{y}$

- **Jacobian-based methods**: Among the infinite solutions, the one choses in the one that mininimizes a suitable (possibly weighted) norm;
- **Null-space methods**: A term is added to the previous solution so as not to affect execution ot the task trajectory, i.e. that beloing to the null space $N(J(q))$
- **Task augmentation methods**: redundancy is reduced/eliminated by adding $s \leq n - m$ further auxiliary tasks (when $s = n - m$, the problem has been "squared")

※ Appendix

3.1 Euler Integration Method

The Euler Integration Method is a first-order numerical procedure for solving ordinary differential equations (ODEs) with a given initial value. Being a first-order method, the local error (error per step) is proportional to the square of the step size, and the global error (error at a given time) is proportional to the step size.

3.1.1 How it works

Consider the problem of calculating the shape of an unknown continuous curve $y(t)$ which starts at a given initial point $y(t_0) = y_0$ and satisfies a given differential equation of the form:

$$\dot{y}(t) = f(t, y(t)) \tag{8}$$

The function $f(t, y(t))$ can be thought of as a formula that provides the slope of the tangent line to the curve at any computed point.

While the exact analytical expression of the curve is initially unknown, its starting point $A_0 = (t_0, y_0)$ is known. Using the differential equation, we can compute the slope of the tangent line at A_0. If we take a sufficiently small time step forward, denoted by Δt, the slope of the true curve does not change significantly. Therefore, we can move along the straight tangent line to find the next point $A_1 = (t_1, y_1)$, where $t_1 = t_0 + \Delta t$.

Assuming the new point A_1 is still an acceptable approximation of the true curve, we can repeat the same reasoning: we compute the new slope at A_1, take another small step Δt along the new tangent line, and find A_2.

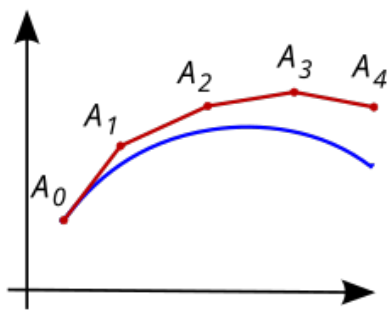


Figure 2: Didascalia

After several iterations, this process generates a polygonal curve $(A_0, A_1, A_2, \dots)$ that approximates the original unknown function. Mathematically, the forward projection rule at each discrete step k is expressed as:

$$y_{k+1} = y_k + \Delta t \cdot f(t_k, y_k) \tag{9}$$

where y_{k+1} is the estimated value at time $t_{k+1} = t_k + \Delta t$. In general, this polygonal curve does not diverge too far from the exact solution, and the integration error can be minimized by choosing a suitably small step size Δt.

3.1.2 Application to Robotic Kinematics

In the context of the *local methods* for redundancy resolution discussed in the main text, the Euler method bridges the gap between differential kinematics and actual joint positions.

At each control cycle, the optimization algorithm provides the optimal joint velocities $\dot{\mathbf{q}}(t_k)$ based on the current state. In this scenario, our ODE is simply $\dot{\mathbf{q}} = f(t, \mathbf{q})$. By applying the Euler integration step using the controller's sampling time T_s as the step size, the low-level controller can compute the reference joint positions to be sent to the motors for the next instant:

$$\mathbf{q}((k+1)T_s) = \mathbf{q}(kT_s) + T_s \dot{\mathbf{q}}(kT_s) \tag{10}$$

This straightforward projection allows the robotic system to seamlessly execute the computed velocities in real time.

References

[1] B. Siciliano, L. Villani, G. Oriolo, A. De Luca. *Foundations of Robotics*. Springer, September 2025.

[2] A. De Luca. "Robotics 2 Course Material and Slides". Sapienza University of Rome, Academic Year 2025/2026. https://www.diag.uniroma1.it/~deluca/rob2_en.php.